

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA1213	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	25
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	M.Naveen	Reg.No.:	192411134
Department:	BE CSE	Date:	

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

Aim

To design and simulate a digital system that performs register transfer operations and basic arithmetic/logic operations (addition, subtraction, AND, OR) using multiple registers, and to analyze the sequence of microoperations and the role of the ALU in processing data.

Tools Used

- Logisim Evolution / Digital Works (logic simulator)
- C / Python (software-level register-transfer simulation)

Theory

A digital computer consists of a set of registers (R0, R1, R2, R3 ...) connected through common data paths to an

Arithmetic and Logic Unit (ALU). Data movement between registers is expressed using Register Transfer Language (RTL), e.g. $R1 \leftarrow R2$ denotes that the content of R2 is transferred to R1. The ALU accepts operands from source registers, performs the operation specified by the control signals, and places the result in a destination register.

Algorithm

- Step 1: Initialize registers R0, R1, R2 and R3 with input data values.
- Step 2: Select two source registers as ALU operands based on the control signal.
- Step 3: Decode the operation code (ADD, SUB, AND, OR) using the control unit.
- Step 4: Route the operand values to the ALU inputs through the common bus.
- Step 5: Perform the selected operation inside the ALU.
- Step 6: Transfer the ALU result back to the destination register on the next clock pulse.

Observation Table

Operation	R0	R1	R2	Result (R3)
ADD (R0+R1)	12	5	8	17
SUB (R0-R2)	12	5	8	4
AND (R0&R1)	12	5	8	4
OR (R0 R2)	12	5	8	12

Result / Analysis

The register transfer system was successfully simulated. Each micro-operation was completed within one clock cycle using the common bus to move data between registers and the ALU.

2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

Aim

To model a computer system using single-bus and multi-bus organization, perform data transfer between registers, memory and I/O, and compare the time taken and efficiency of the two approaches.

Tools Used

- Logisim Evolution (bus and multiplexer modelling)
- Python (cycle-count simulation of bus transfers)

Theory

In a single-bus organization, all registers, the ALU, memory and I/O units share one common bus, so only one data transfer can occur at any instant; simultaneous transfers must be serialized, increasing the number of clock cycles needed for complex operations. In a multi-bus organization (e.g. a 3-bus structure with two source buses and one destination bus), multiple operands can be read from different registers.

Algorithm

- Step 1: Model registers, ALU, memory and I/O as modules connected to the bus(es).
- Step 2: For single-bus design, allow only one source-to-destination transfer per clock cycle.
- Step 3: For multi-bus design, allow two operand reads (bus A, bus B) and one result write (bus C) in the same cycle.
- Step 4: Execute a fixed instruction sequence (e.g. $R1 \leftarrow \text{Mem}$, $R2 \leftarrow \text{Mem}$, $R3 \leftarrow R1 + R2$, $\text{Mem} \leftarrow R3$) on both models.

Observation Table

Organization	Buses	Cycles Taken	Parallel Transfers
Single-Bus	1	4	No
Multi-Bus	3	2	Yes (2 reads + 1 write/cycle)

Result / Analysis

The single-bus model required 4 clock cycles to complete the instruction sequence since every transfer had to wait for the bus to be free. The multi-bus model completed the same

sequence in 2 cycles by reading both operands simultaneously on separate buses and writing the ALU result on a third bus, giving roughly a 50% reduction in execution time. This confirms that multiple buses reduce bottlenecks and improve parallel data transfer, though they require more control logic and wiring than a single-bus design.

3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

Aim

To simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back), measure speedup and throughput, and compare its performance with a non-pipelined system.

Tools Used

- Python (pipeline timing-diagram simulator)
- MARS / Ripes (MIPS pipeline visual simulator) — optional

Theory

Pipelining overlaps the execution of multiple instructions by dividing instruction processing into independent stages — Fetch (IF), Decode (ID), Execute (EX), Memory access (MEM) and Write-back (WB). While one instruction is being decoded, the next can be fetched, so

Algorithm

- Step 1: Define $k = 5$ pipeline stages and n instructions to execute.
- Step 2: For the non-pipelined model, compute total cycles = $n \times k$.
- Step 3: For the pipelined model, compute total cycles = $k + (n - 1)$, assuming no stalls.
- Step 4: Build a stage-vs-cycle timing diagram showing each instruction moving through
IF→ID→EX→MEM→WB.

Timing Diagram ($n = 5$ instructions, $k = 5$ stages)

Instr	C1	C2	C3	C4	C5	C6	C7	C8	C9
-------	----	----	----	----	----	----	----	----	----

I1	IF	ID	EX	MEM	WB				
I2		IF	ID	EX	MEM	WB			
I3			IF	ID	EX	MEM	WB		
Instr	C1	C2	C3	C4	C5	C6	C7	C8	C9
I4				IF	ID	EX	MEM	WB	
I5					IF	ID	EX	MEM	WB

Result / Analysis

For 5 instructions, the non-pipelined processor requires 25 cycles (5×5), while the pipelined processor completes execution in only 9 cycles ($5 + 4$), giving a speedup of about 2.78 and a throughput approaching 1 instruction per cycle in steady state.

4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

Aim

To develop a pipeline execution scenario that exhibits data, control and structural hazards, observe their impact on instruction execution, and implement stalling, data forwarding and branch prediction to resolve them.

Tools Used

- Python (hazard-detection and stall simulator)
- Ripes / MARS (visual pipeline simulator) — optional

Theory

- Structural Hazard: occurs when two instructions need the same hardware resource (e.g. a single memory port) in the same cycle.
- Data Hazard: occurs when an instruction depends on the result of a previous instruction that has not yet completed (e.g. Read-After-Write).

- Control Hazard: occurs when the pipeline fetches instructions before a branch outcome is known, potentially fetching the wrong instructions.

Algorithm

- Step 1: Take an instruction sequence containing a data dependency, e.g. ADD R1,R2,R3 followed by SUB R4,R1,R5.
- Step 2: Detect the RAW hazard by comparing source registers of each instruction with destination registers of the two preceding instructions.
- Step 3: Without forwarding — insert stall (bubble) cycles until the dependent value is written back to the register file.
- Step 4: With forwarding — bypass the ALU output of the producing instruction directly to the EX stage of the consuming instruction, eliminating most stalls.
- Step 5: For a branch instruction, predict the outcome (e.g. predict 'not taken'); if the prediction is wrong, flush the incorrectly fetched instructions and restart fetch from the correct target.

Observation Table

Hazard Type	Cause	Resolution Technique	Stall Cycles Saved
Structural	Shared memory port for IF & MEM	Separate instruction/data memory or extra port	1 cycle/instr
Data (RAW)	SUB depends on ADD result	Data Forwarding	2 cycles/instr
Control	Branch outcome unknown at fetch	Branch Prediction	1–2 cycles/branch

Result / Analysis

Without any hazard-handling, the dependent SUB and AND instructions each required 2 stall cycles to wait for the register write-back, giving 4 bubble cycles overall. Introducing data forwarding eliminated these stalls completely by routing the ALU result directly to the next instruction's execute stage.

5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Aim

To analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and to extend the study to hyper-threading and simultaneous multithreading (SMT), evaluating instruction throughput and resource utilization.

Tools Used

- Python (issue-width / IPC calculator)
- gem5 / Tomasulo simulator (conceptual reference)

Theory

- Superscalar Execution: the processor has multiple parallel execution units and can fetch, decode and issue more than one instruction per clock cycle (issue width > 1).
- Out-of-Order Execution: instructions are executed as soon as their operands are ready rather than strictly in program order, using techniques such as Tomasulo's algorithm and reservation stations, then results are committed in order (in-order retirement).
- Speculative Execution: the processor predicts the outcome of branches (or other uncertain events) and executes instructions beyond them speculatively; correct predictions improve throughput while mispredictions require squashing speculative results.

Algorithm

- Step 1: Define the issue width (number of instructions the processor can issue per cycle).
- Step 2: For a scalar (1-wide) processor, compute cycles = $n \text{ instructions} / 1$.
- Step 3: For a superscalar processor of width w , compute ideal cycles = n / w (assuming no dependency stalls).
- Step 4: Model out-of-order issue by allowing independent instructions to be scheduled ahead of a stalled instruction, using a small reservation-station queue.

Observation Table

Configuration	Issue Width	Cycles (n=12)	IPC	Resource Utilization
---------------	-------------	---------------	-----	----------------------

Scalar (in-order)	1	12	1.00	Low
Superscalar (inorder)	4	3	4.00 (ideal)	Medium
Superscalar + OoO	4	2	6.00	High
Superscalar + OoO + SMT (2T)	4	2 (2 threads)	~6.00 per thread- slot	Highest

Result / Analysis

The scalar processor required 12 cycles to issue 12 instructions (IPC = 1). Widening the pipeline to a 4-issue superscalar design reduced this to an ideal 3 cycles, but real dependencies prevent the full width from being used every cycle. Allowing out-of-order issue let independent instructions fill slots that would otherwise stall, further reducing effective cycles and raising IPC.

Code of Conduct:

I M.Naveen / 192411134(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: naveen

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation